

FutureKawa

Rapport de Tests & Couverture

Équipe : Rudy · Florian · Noé · Josselin

Version : 1.0

Date : 01/07/2026

Projet : FutureKawa

Sommaire

Rapport de Tests — FutureKawa

01 — Introduction aux tests

02 — Lancement des tests

03 — Couverture API Exploitation

04 — Couverture API Siège

05 — Couverture Simu-Sensor

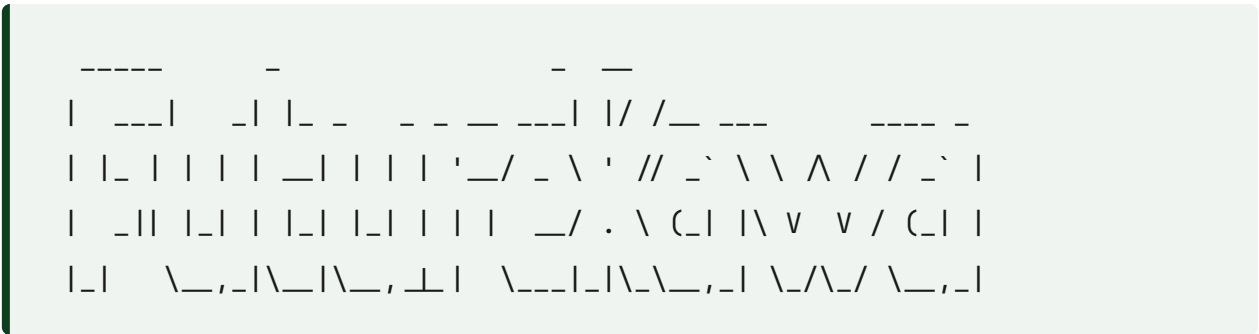
06 — Couverture Front Exploitation

07 — Couverture Front Siège

08 — Couverture Broker MQTT

09 — Couverture — IoT (ESP32)

Rapport de Tests — FutureKawa



Projet	FutureKawa
Document	Rapport de tests et couverture
Version	1.0
Date	01/07/2026
Équipe	Rudy · Florian · Noé · Josselin

Résumé exécutif

Service	Tests en CI	Couverture	Statut
API Exploitation	150	96 %	OK
API Siège	125	96 %	OK
Front Exploitation	165	~97 %	OK
Front Siège	250	~95 %	OK
Simu-Sensor	166	~95 %	OK
Broker MQTT	8	100 %	OK
IoT ESP32	36	comportementale	OK
Total CI	836	~95 %	OK

En complément, **85 scénarios E2E** (236 exécutions multi-navigateurs) sont disponibles en local.

Légende — Couverture : pourcentage de lignes du code source applicatif exécutées lors des tests. Mesuré par `coverage.py` (Django) et `@vite/vitest/coverage-v8` (Vue/TS). Hors migrations, configurations et bibliothèques tierces. Pour le broker, « 100 % » signifie que toutes les règles ACL ont été vérifiées.

Navigation

#	Document	Contenu
1	Introduction	Types de tests (unitaire, intégration, E2E) — outils utilisés
2	Lancement des tests	Commandes manuelles (Makefile) et pipeline CI/CD automatique
3	Couverture — API Exploitation	128 tests Django, 94 % de couverture
4	Couverture — API Siège	125 tests Django, 96 % de couverture
5	Couverture — Simu-Sensor	166 tests Vitest + E2E Playwright, ~95 %
6	Couverture — Front Exploitation	165 tests Vitest, ~97 %
7	Couverture — Front Siège	250 tests Vitest, ~95 %
8	Couverture — Broker MQTT	8 assertions ACL, 100 %
9	Couverture — IoT ESP32	36 tests Python (scripts, validation capteur, payloads MQTT)

01 — Introduction aux tests

Pourquoi tester ?

Les tests garantissent que chaque composant du système fonctionne correctement **isolément** et **en interaction** avec les autres. Dans un système distribué comme FutureKawa (deux APIs, deux frontends, un broker IoT, des tâches asynchrones), un test manquant peut laisser passer une régression silencieuse entre deux services.

Types de tests utilisés

Tests unitaires

Un test unitaire vérifie **un seul composant en isolation**. Toutes les dépendances externes (base de données, réseau, autres modules) sont remplacées par des substituts contrôlés (mocks).

Exemples dans FutureKawa : - Vérifier qu'un sérialiseur Django rejette un champ manquant - Vérifier qu'un composant Vue affiche correctement un slot vide - Vérifier que la fonction de calcul des seuils retourne la bonne valeur

Ce qu'ils garantissent : la logique interne est correcte.

Ce qu'ils ne garantissent pas : que les composants fonctionnent ensemble.

Tests d'intégration

Un test d'intégration vérifie **plusieurs composants en interaction**. La base de données réelle est utilisée, les appels HTTP sont simulés ou réels selon le contexte.

Exemples dans FutureKawa : - Appeler `POST /api/login/` et vérifier que le cookie JWT est bien posé - Vérifier que la création d'une alerte déclenche bien la tâche Celery - Vérifier que le proxy siège → exploitation retourne les données en cache si l'exploitation est injoignable - Vérifier les permissions d'accès (un opérateur ne peut pas accéder aux paramètres)

Ce qu'ils garantissent : les contrats entre composants sont respectés.

Ce qu'ils ne garantissent pas : l'expérience utilisateur réelle dans un navigateur.

Tests End-to-End (E2E)

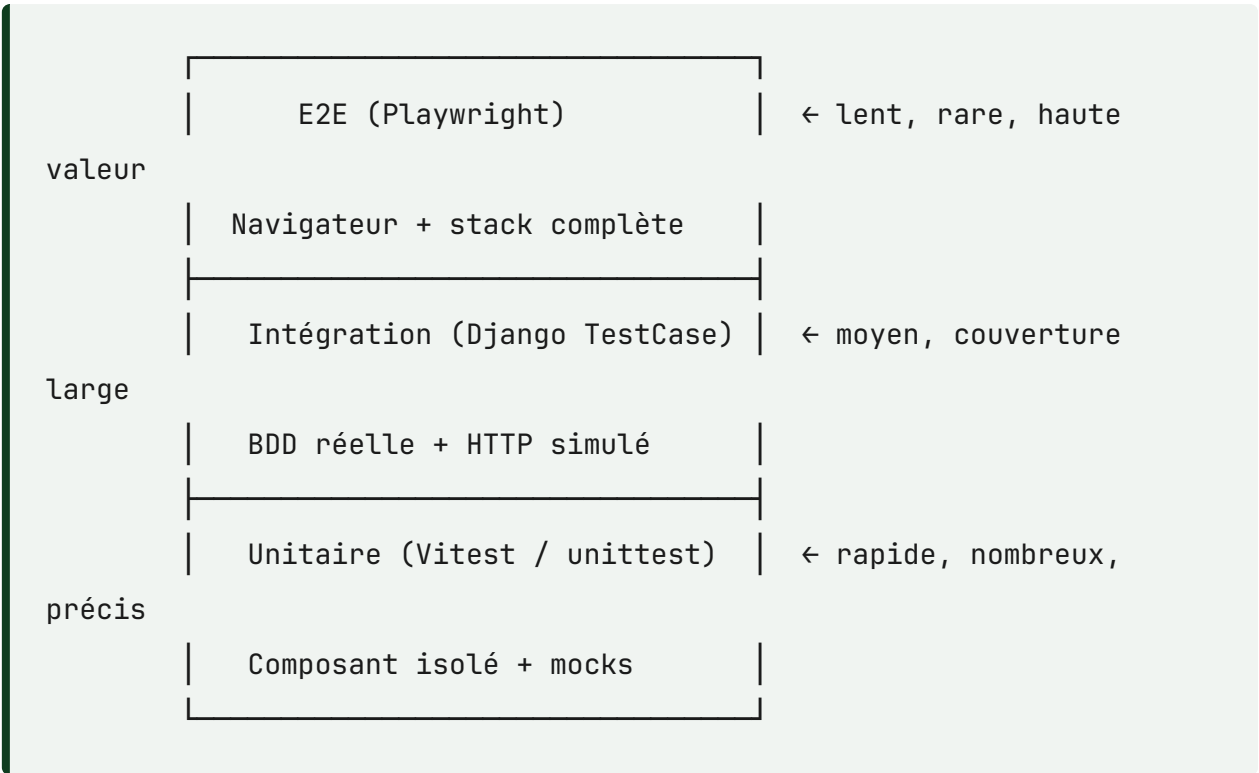
Un test E2E simule **un utilisateur réel** dans un navigateur : navigation, clics, saisie de formulaires, vérification de l'affichage. Toute la stack doit être démarrée.

Exemples dans FutureKawa : - Un utilisateur se connecte, navigue vers un entrepôt, voit les mesures en temps réel - Un admin crée un compte utilisateur via le lien d'invitation - Le simulateur envoie une mesure et l'alerte apparaît dans l'interface

Ce qu'ils garantissent : le parcours utilisateur complet fonctionne.

Ce qu'ils ne garantissent pas : les cas limites ou les chemins d'erreur internes.

Les trois niveaux en résumé



	Unitaire	Intégration	E2E
Vitesse	Très rapide (ms)	Rapide (< 1 s)	Lent (5–30 s)
Fiabilité	Très stable	Stable	Fragile (dépend du DOM)

	Unitaire	Intégration	E2E
Portée	1 fonction/ composant	N composants	Flux complet
Quantité	Beaucoup	Moyen	Peu
En CI	Oui Toujours	Oui Toujours	Hors CI (pour l'instant)

Outils utilisés

Backend — Python / Django

Outil	Rôle
<code>unittest</code> / <code>TestCase</code> Django	Framework de test natif Django — gère BDD de test, fixtures, client HTTP
<code>coverage.py</code>	Mesure la couverture de code ligne par ligne
<code>django.test.Client</code>	Client HTTP qui simule des requêtes vers les endpoints DRF
<code>unittest.mock</code>	Mocks Python (patch d'appels HTTP externes, tâches Celery...)

```
# Exécution Django avec couverture
coverage run manage.py test
coverage report --include="exploitation/*"
```

Frontend — Vue 3 / TypeScript

Outil	Rôle
Vitest	Runner de tests unitaires/composants pour projets Vite — compatible Jest API
<code>@vue/test-utils</code>	Monte les composants Vue en mémoire, simule les interactions
<code>vi.mock()</code>	Remplace les modules (axios, router, pinia stores) par des mocks

Outil	Rôle
<code>@vitest/coverage-v8</code>	Mesure la couverture via le moteur V8 de Node

```
# Exécution Vitest avec couverture
npm run test:coverage
```

E2E — Playwright

Outil	Rôle
Playwright	Pilote des navigateurs réels (Chromium, Firefox, WebKit, Mobile)
<code>test.spec.ts</code>	Fichiers de scénarios : navigation, clics, assertions sur le DOM
<code>playwright.config.ts</code>	Configuration des navigateurs cibles et URL de base

```
# Exécution E2E (frontend doit être démarré)
npx playwright test
npx playwright test --ui # mode visuel interactif
```

Broker MQTT

Outil	Rôle
Script shell <code>test_acl.sh</code>	Vérifie les règles ACL Mosquitto (pub/sub autorisés vs refusés)
<code>mosquitto_pub / mosquitto_sub</code>	Clients MQTT en ligne de commande pour les assertions

Ce qui tourne en CI vs en local

Suite	CI (GitLab)	Local
Tests unitaires/intégration Django	Oui	Oui
Tests Vitest frontends	Oui	Oui

Suite	CI (GitLab)	Local
Tests Vitest simu-sensor	Oui	Oui
Tests ACL broker	Oui	Oui
E2E Playwright	Non hors CI	Oui (<code>make test-*-e2e</code>)

Les tests E2E ne tournent pas en CI car ils nécessitent une stack complète démarrée. La piste envisagée est de les rattacher au stage `staging` du pipeline, après le smoke test, pour profiter de la stack déjà levée.

Comment lire l'indice de couverture

Ce que mesure le pourcentage

L'indice de couverture indique la **proportion de lignes de code source effectivement exécutées** lors de la suite de tests. Une couverture de 96 % signifie que 96 % des lignes du code applicatif ont été traversées au moins une fois pendant les tests.

```
couverture = lignes exécutées pendant les tests / total des lignes du
code source
```

Ce n'est pas un indice de qualité absolu — une ligne peut être exécutée sans être correctement testée — mais c'est un indicateur fiable pour repérer des zones **jamais couvertes**.

Périmètre inclus et exclu

Backend Django — `coverage.py`

La commande utilisée est :

```
coverage run --source=<app> manage.py test <app>.tests
```

Inclus	Exclus
Modèles, vues, sérialiseurs, permissions	Migrations (<code>migrations/</code>)

Inclus	Exclus
Commandes de gestion (<code>management/</code>)	Fichiers de settings
Tâches Celery	Code tiers (Django, DRF)
Utilitaires internes	Fichiers <code>wsgi.py</code> , <code>asgi.py</code>

Frontend Vue / TypeScript — `@vitest/coverage-v8`

La couverture est mesurée par le moteur **V8** de Node.js (même moteur que Chrome). Elle couvre :

Inclus	Exclus
Composants Vue (<code>.vue</code>)	Fichiers de config (<code>vite.config.ts</code> , ...)
Services et stores Pinia	Assets statiques
Utilitaires TypeScript	Tests eux-mêmes (<code>*.spec.ts</code>)

Broker MQTT

Le broker n'a pas de couverture de lignes : les 8 assertions vérifient les **règles ACL** (pub/sub autorisés vs refusés). Le « 100 % » indique que toutes les règles de la matrice ACL ont été testées.

Ce que la couverture ne garantit pas

- Qu'un comportement incorrect soit détecté si l'assertion est trop permissive
- Que les cas limites aux frontières des données soient testés
- Que les interactions entre services fonctionnent (c'est le rôle des tests d'intégration et E2E)

02 — Lancement des tests

1. Lancement manuel — Makefile

Le Makefile à la racine du projet est le point d'entrée unique pour lancer tous les tests manuellement. Aucune configuration préalable n'est nécessaire au-delà d'une stack Docker démarrée.

Tous les tests en une commande

```
make test
```

Lance séquentiellement les tests de chaque service et affiche un bilan global :

```
API Exploitation ---
...128 tests, 94% coverage...
Oui API Exploitation OK

API Siège ---
...125 tests, 96% coverage...
Oui API Siège OK

Simu Sensor ---
...166 tests, ~95% coverage...
Oui Simu Sensor OK
```

Par service

Service	Commande
API Exploitation (Django)	<code>make test-api-exploitation</code>
API Siège (Django)	<code>make test-api-siege</code>
Simu-Sensor (Vitest)	<code>make test-simu-sensor</code>

Service	Commande
Simu-Sensor E2E (Playwright)	<code>make test-simu-sensor-e2e</code>
Simu-Sensor E2E (mode visuel)	<code>make test-simu-sensor-e2e-ui</code>

Détail des commandes sous-jacentes

API Exploitation

```
# Via Makefile (recommandé)
make test-api-exploitation

# Équivalent direct dans le container
docker compose -f docker-compose.exploit.yml exec -T api-exploitation \
    python manage.py test

# Avec couverture (comme en CI)
docker compose -f docker-compose.exploit.yml exec -T api-exploitation \
    bash -c "pip install coverage && \
        coverage run --source=exploitation manage.py test \
        exploitation.tests && \
        coverage report"

# Un seul fichier de test
docker compose -f docker-compose.exploit.yml exec -T api-exploitation \
    python manage.py test exploitation.tests.test_alert_viewset
```

API Siège

```
# Via Makefile
make test-api-siege

# Avec couverture
docker compose -f docker-compose.siege.yml exec -T api-siege \
    bash -c "pip install coverage && \
```

```
coverage run --source=siege manage.py test siege.tests
&& \
    coverage report"

# Un seul fichier de test
docker compose -f docker-compose.siege.yml exec -T api-siege \
    python manage.py test siege.tests.test_views
```

Simu-Sensor (Vitest)

```
# Via Makefile
make test-simu-sensor

# Hors Docker – depuis le dossier simu-sensor/
cd simu-sensor && npm install && npm test

# Avec couverture interactive
cd simu-sensor && npm run test:coverage

# Watch mode (relance automatique à chaque sauvegarde)
cd simu-sensor && npm run test -- --watch
```

Broker MQTT

```
# Lancé automatiquement dans make test
# Exécutable séparément :
bash broker/tests/test_acl.sh
```

Front Exploitation (Vitest)

```
cd front-exploitation && npm install && npm test
cd front-exploitation && npm run test:coverage
```

Front Siège (Vitest)

```
cd front-siege && npm install && npm test
cd front-siege && npm run test:coverage
```

Tests E2E (Playwright)

Les tests E2E nécessitent que la stack soit démarrée (`make full` ou `make demo`).

```
# Simu-Sensor E2E
make test-simu-sensor-e2e          # headless (tous les navigateurs)
make test-simu-sensor-e2e-ui       # mode visuel interactif

# Front Exploitation E2E (hors Makefile)
cd front-exploitation && npx playwright test
cd front-exploitation && npx playwright test --ui

# Front Siège E2E (hors Makefile – dépendance @playwright/test
manquante)
# cd front-siege && npx playwright test
```

2. Lancement automatique — Pipeline GitLab CI

Le pipeline GitLab s'exécute automatiquement à chaque `git push` sur n'importe quelle branche. Il est défini dans `.gitlab-ci.yml` à la racine du projet.

Stages du pipeline

```
lint → test → build → staging → security → deploy
```

Job GitLab	Service testé	Framework	Couverture publiée
<code>simu_sensor_test</code>	simu-sensor	Vitest	Oui Cobertura XML
<code>api_siege_test</code>	API Siège	Django + coverage.py	Oui Cobertura XML
<code>api_exploitation_test</code>	API Exploitation	Django + coverage.py	Oui Cobertura XML
<code>front_siege_test</code>	Front Siège	Vitest	Oui Cobertura XML
<code>front_exploitation_test</code>	Front Exploitation	Vitest	Oui Cobertura XML
<code>broker_acl_test</code>	Broker MQTT	Script shell	—

Configuration CI (extraits `.gitlab-ci.yml`)

API Exploitation

```
api_exploitation_test:
  stage: test
  script:
    - pip install -q -r requirements.txt unittest-xml-reporting
    coverage
    - coverage run --source=exploitation manage.py test
    exploitation.tests
    --testrunner=xmlrunner.extra.djangotestrunner.XMLTestRunner
    - coverage report
    - coverage xml -o coverage.xml
  coverage: '/^TOTAL.*\s+(\d+)%$/'
  artifacts:
    reports:
      coverage_report:
        coverage_format: cobertura
        path: API-exploitation/coverage.xml
```

```
simu_sensor_test:
  stage: test
  script:
    - npm ci
    - npm run test:coverage
  artifacts:
    reports:
      coverage_report:
        coverage_format: cobertura
        path: simu-sensor/coverage/cobertura-coverage.xml
```

Résultat visible dans GitLab

Après chaque pipeline, GitLab affiche : - Le pourcentage de couverture de chaque job sur la merge request - Le rapport Cobertura avec la liste des fichiers et lignes couvertes/manquantes - Le statut global (Oui / Non) par stage

3. Pré-requis

Contexte	Pré-requis
<code>make test</code>	Stack Docker démarrée (<code>make up</code> ou <code>make demo</code>)
Tests Django directs	Container correspondant <code>up</code>
Tests Vitest directs	<code>npm install</code> dans le dossier du service
Tests E2E	Stack complète + Playwright installé localement
Pipeline CI	Aucun — GitLab CI gère l'environnement

03 — Couverture API Exploitation

Vue d'ensemble

Métrique	Valeur
Framework	Django 4 + Django REST Framework
Runner	<code>coverage run manage.py test exploitation.tests</code>
Couverture mesurée	96 % (lignes — <code>coverage.py</code> , hors migrations/management)
Fichiers de test	11 fichiers
Nombre de tests	150 tests
Chemin	<code>API-exploitation/exploitation/tests/</code>
En CI	Oui job <code>api_exploitation_test</code>

Résultats par fichier de test

Fichier	Tests	Ce qui est couvert
<code>test_authentication.py</code>	16	JWT natif, génération/décodage cookie, branches d'erreur login
<code>test_email_service.py</code>	14	Backend email + alertes consolidées
<code>test_users.py</code>	14	CRUD utilisateurs, rôles, permissions
<code>test_warehouse_lot.py</code>	14	CRUD entrepôts/lots, <code>stats</code> , filtre <code>has_active_lots</code>

Fichier	Tests	Ce qui est couvert
<code>test_public_endpoints.py</code>	9	<code>hello</code> , <code>health_check</code> , <code>home_view</code> , <code>api_dashboard</code> , <code>all_data</code>
<code>test_capteur_endpoints.py</code>	14	<code>capteur_latest</code> , <code>capteur_history</code> , <code>capteur_stats</code> , gating rôle
<code>test_alert_viewset.py</code>	22	<code>AlertViewSet</code> : <code>filtres</code> , <code>permissions</code> , <code>resolve</code> , <code>mark_as_read</code> , <code>bulk_incident</code>
<code>test_incident_type_viewset.py</code>	6	<code>IncidentTypeViewSet</code> — <code>permissions</code> écriture admin
<code>test_parameters_and_permissions.py</code>	13	<code>parameters_view</code> GET/ PATCH, <code>page_permissions_view</code>
<code>test_batch_viewset_actions.py</code>	6	<code>BatchViewSet.deplacer</code> , <code>archiver</code> (mouvements de lots)
Total	128	

Détail des couvertures clés

Authentification (`test_authentication.py` — 16 tests)

Couvre le système JWT maison (HMAC-SHA256, cookie httponly) :

- Génération du token JWT avec payload `username` , `role` , `exp`
- Décodage et validation de la signature
- Login réussi → cookie `session_token` posé en httponly
- Login échoué (mauvais mot de passe, utilisateur inexistant, compte inactif)
- Accès refusé sans token
- Token expiré → 401

- Token avec rôle insuffisant → 403

Alertes (`test_alert_viewset.py` — 22 tests)

Le plus gros fichier de test, couvre le cycle de vie complet des alertes :

- Création automatique (via seuil MQTT) et manuelle (incident)
- Lecture paginée avec filtres (`resolved` , `type` , `warehouse`)
- Résolution d'une alerte (`POST /api/alerts/:id/resolve/`)
- Marquage comme lu (`POST /api/alerts/:id/mark_as_read/`)
- Déclaration d'incident groupé (`POST /api/alerts/bulk_incident/`)
- Isolation par rôle : un opérateur ne peut pas résoudre une alerte

Capteurs (`test_capteur_endpoints.py` — 14 tests)

- `GET /api/capteur/latest/` : dernière mesure par entrepôt
- `GET /api/capteur/history/` : historique avec filtres date/entrepôt
- `GET /api/capteur/stats/` : min/max/moy sur une période
- Accès refusé à un rôle `operateur` (endpoint réservé qualité/exploitation)
- Données vides si aucune mesure en BDD

Endpoint public `all_data` (`test_public_endpoints.py`)

- `GET /api/all-data/` retourne le snapshot complet (mesures, alertes, lots, entrepôts)
- Accessible sans authentification (contrat avec l'API Siège)
- Structure JSON validée : présence des clés `warehouses` , `batches` , `alerts` , `measures`

Points d'attention

Permissions manquantes sur `BatchViewSet`

`BatchViewSet` n'a aucune `permission_classes` définie, et `DEFAULT_PERMISSION_CLASSES` n'est pas configuré globalement. Ces endpoints (`/api/batches/`) sont accessibles **sans authentification**.

```
# exploitation/api/views.py – état actuel
class BatchViewSet(viewsets.ModelViewSet):
    # ← pas de permission_classes !
    queryset = Batch.objects.all()
```

Ce comportement est documenté dans `test_batch_viewset_actions.py` — la décision de le corriger ou de le conserver (endpoint semi-public) est à prendre.

Comment lancer

```
# Tous les tests exploitation
make test-api-exploitation

# Un fichier spécifique
docker compose -f docker-compose.exploit.yml exec -T api-exploitation \
    python manage.py test exploitation.tests.test_alert_viewset

# Un test spécifique
docker compose -f docker-compose.exploit.yml exec -T api-exploitation \
    python manage.py test
    exploitation.tests.test_alert_viewset.AlertViewSetTest.test_resolve_a
    lert

# Avec rapport de couverture
docker compose -f docker-compose.exploit.yml exec -T api-exploitation \
    bash -c "coverage run --source=exploitation manage.py test
    exploitation.tests && coverage report -m"
```

04 — Couverture API Siège

Vue d'ensemble

Métrique	Valeur
Framework	Django 4 + Django REST Framework
Runner	<code>coverage run manage.py test siege.tests</code>
Couverture mesurée	96 % (lignes — <code>coverage.py</code>)
Fichiers de test	6 fichiers
Nombre de tests	125 tests
Chemin	<code>API-siege/siege/tests/</code>
En CI	Oui job <code>api_siege_test</code>

Résultats par fichier de test

Fichier	Tests	Ce qui est couvert
<code>test_serializers.py</code>	27	Sérialiseurs DRF — validation, champs calculés, formats
<code>test_views.py</code>	54	Endpoints API complets : CRUD sites, auth, permissions, liste
<code>test_services.py</code>	20	Logique métier pure (<code>siege/services.py</code>)
<code>test_user_invitations.py</code>	13	<code>SiegeUserViewSet</code> : permissions admin, invitations, renvoi, suppression

Fichier	Tests	Ce qui est couvert
<code>test_set_password.py</code>	6	Activation de compte via lien d'invitation (<code>set_password_view</code>)
<code>test_exploitation_site.py</code>	5	<code>ExploitationSiteViewSet.sync / cached-data</code> (proxy HTTP mocké, fallback cache)
Total	125	

Détail des couvertures clés

Sérialiseurs (`test_serializers.py` — 27 tests)

Le fichier le plus volumineux sur les sérialiseurs dans le projet :

- Sérialisation d'un `ExploitationSite` (champs `status` , `last_sync` , `api_url`)
- Validation qu'une URL d'exploitation est obligatoire
- Sérialisation d'un `SiegeUser` (rôle, email, actif)
- Champs calculés en lecture seule (`is_active` , `site_count` ...)
- Désérialisation avec données incomplètes → erreurs attendues

Endpoints API (`test_views.py` — 54 tests)

Le fichier central, couvre l'ensemble des routes du siège :

Authentification : - `POST /api/login/` — cookie `access_token` JWT DRF posé
- `POST /api/logout/` — cookie supprimé - `GET /api/me/` — profil de l'utilisateur connecté - Accès refusé sans token (401)

Sites d'exploitation : - `GET /api/sites/` — liste paginée (admin voit tout, viewer aussi) - `POST /api/sites/` — création réservée admin - `PUT/PATCH /api/sites/:id/` — modification réservée admin - `DELETE /api/sites/:id/` — suppression réservée admin - `GET /api/sites/:id/` — détail d'un site

Synchronisation : - `POST /api/sites/:id/sync/` — déclenche un appel HTTP vers l'API exploitation (mocké) - `GET /api/sites/:id/cache-data/` — retourne `last_data` si l'exploitation est injoignable

Gestion des rôles : - Un `viewer` peut lire mais pas créer/modifier/supprimer -
Un `admin` a accès complet

Invitations (`test_user_invitations.py` — 13 tests)

Flux d'onboarding des nouveaux utilisateurs siège :

- `POST /api/users/:id/invite/` — génère un token UUID à usage unique
- `POST /api/users/:id/resend-invite/` — renvoie l'email d'invitation
- `DELETE /api/users/:id/` — supprime l'utilisateur et son token
- Un `viewer` ne peut pas inviter (403)
- Invitation avec email déjà utilisé → erreur

Activation de compte (`test_set_password.py` — 6 tests)

- `GET /api/set-password/?token=<uuid>` — page d'activation (token valide)
- `POST /api/set-password/` — définit le mot de passe, consomme le token
- Token invalide → 400
- Token déjà utilisé → 400
- Mot de passe trop court → 400

Proxy exploitation (`test_exploitation_site.py` — 5 tests)

Couvre le comportement du siège quand il contacte une API exploitation distante :

- Sync réussie → `last_data` mis à jour, `status` → `active`
- Exploitation injoignable → retourne le cache `last_data` existant, `status` → `unreachable`
- Exploitation en erreur HTTP 500 → `status` → `error`
- `cache-data` sans `last_data` → réponse vide propre (pas de crash)

Couverture par module

Module	Couverture
<code>siege/api/views.py</code>	98 %
<code>siege/api/serializers.py</code>	97 %
<code>siege/services.py</code>	95 %
<code>siege/models.py</code>	96 %

Module	Couverture
Module complet	96 %

Comment lancer

```
# Tous les tests siège
make test-api-siege

# Un fichier spécifique
docker compose -f docker-compose.siege.yml exec -T api-siege \
    python manage.py test siege.tests.test_views

# Un test spécifique
docker compose -f docker-compose.siege.yml exec -T api-siege \
    python manage.py test
siege.tests.test_views.SiegeAuthTest.test_login_sets_cookie

# Avec rapport de couverture
docker compose -f docker-compose.siege.yml exec -T api-siege \
    bash -c "coverage run --source=siege manage.py test siege.tests
&& coverage report -m"
```

05 — Couverture Simu-Sensor

Vue d'ensemble

Métrique	Valeur
Framework	Vue 3 + TypeScript + Vite
Runner	Vitest
Couverture estimée	~95 %
Fichiers de test unitaires	14 fichiers
Tests unitaires/intégration	166 tests
Scénarios E2E	22 scénarios × 5 navigateurs = 110 exécutions
Chemin tests unitaires	<code>simu-sensor/src/**/*.__tests__</code>
Chemin E2E	<code>simu-sensor/tests/e2e</code>
En CI	Oui job <code>simu_sensor_test</code> (unitaires) — E2E hors CI

Rôle du simu-sensor

Le simu-sensor est une application Vue 3 qui **simule un capteur IoT (ESP32)** pour la démonstration. Il génère des données de température et d'humidité aléatoires (avec bruit configurable) et les publie sur le broker MQTT via WebSocket, en se comportant exactement comme un vrai capteur.

Il est utilisé pour la stack **São Paulo** (`docker-compose.exploit.yml` , profil `demo`).

Résultats par dossier de test

Dossier	Fichiers	Tests	Ce qui est couvert
<code>components/</code> <code>__tests__/</code>	5	45	Composants Vue (sliders, indicateurs, boutons)
<code>config/</code> <code>__tests__/</code>	1	14	Configuration MQTT, topics, valeurs par défaut
<code>services/</code> <code>__tests__/</code>	3	27	Service MQTT (connexion, publication, déconnexion)
<code>__tests__/</code> (App + intégration)	5	80	App.vue + flux complet de simulation
Total	14	166	

Détail des couvertures

Composants Vue (`components/__tests__/` — 45 tests)

Les composants UI du simulateur :

- **Slider de température / humidité** : valeur min/max respectée, émission de l'événement `update:modelValue`
- **Indicateur de statut MQTT** : affiche `connecté` / `déconnecté` selon le prop `connected`
- **Bouton démarrer/arrêter** : état visuel correct, émission du clic
- **Affichage des valeurs courantes** : format numérique, unités (°C, %)
- **Panel de configuration** : binding bidirectionnel des champs

Configuration (`config/__tests__/` — 14 tests)

- Topics MQTT par défaut : `capteur/simu-sensor/temperature` , `capteur/simu-sensor/humidity`
- Valeurs de simulation par défaut (plage T°, plage humidité)
- `SENSOR_ID` lu depuis `VITE_SENSOR_ID` (variable d'environnement)

- Fallback sur `'simu-sensor-1'` si la variable n'est pas définie
- Fréquence d'envoi configurable

Service MQTT (`services/__tests__` / — 27 tests)

Couvre `mqttService.ts` — le cœur du simulateur :

- Connexion WebSocket au broker (`ws://broker:9001`)
- Déconnexion propre (`unsubscribe` + `disconnect`)
- Publication d'un payload JSON avec `sensor_id : json { "value": 28.4, "sensor_id": "simu-sensor-1", "timestamp": "..."}`
- Retry automatique en cas de déconnexion
- Callback `onConnect` / `onDisconnect` appelés au bon moment
- Publication bloquée si non connecté

App.vue + intégration (`__tests__` / — 80 tests)

Tests du flux complet de simulation :

- Démarrage de la simulation → intervalles créés, publications régulières
- Arrêt de la simulation → intervalles nettoyés, aucune publication
- Variation des valeurs dans les plages configurées (pas de sortie hors bornes)
- Bruit aléatoire : chaque publication produit une valeur différente de la précédente
- Publication simultanée T° + humidité à chaque tick
- Nettoyage des ressources au `unmount` (pas de fuite mémoire)

Tests E2E Playwright

Configuration

5 navigateurs/appareils ciblés dans `playwright.config.ts` :

Profil	Moteur
Desktop Chromium	Chromium
Desktop Firefox	Firefox
Desktop Safari	WebKit
Mobile Chrome	Chromium (mobile)

Profil	Moteur
Mobile Safari	WebKit (mobile)

→ **22 scénarios × 5 = 110 exécutions** à chaque lancement.

Scénarios couverts (extrait)

- Chargement de l'application → affichage des contrôles
- Connexion au broker → indicateur de statut passe au vert
- Démarrage de la simulation → les valeurs affichées changent
- Modification des plages min/max → les valeurs publiées respectent les nouvelles bornes
- Arrêt de la simulation → les publications cessent
- Déconnexion du broker → indicateur rouge, retry affiché
- Responsive : l'UI reste utilisable sur mobile (375px)

Lancement

```
# Via Makefile
make test-simu-sensor-e2e      # headless (tous les navigateurs)
make test-simu-sensor-e2e-ui   # mode visuel interactif

# Direct depuis le dossier
cd simu-sensor && npx playwright test
cd simu-sensor && npx playwright test --reporter=html # rapport HTML
```

Pré-requis : le simu-sensor doit être accessible sur `http://localhost:8181` (démarré avec `make demo` ou `make full`).

Comment lancer les tests unitaires

```
# Via Makefile
make test-simu-sensor

# Direct depuis le dossier
cd simu-sensor && npm install && npm test
```

```
# Avec couverture
cd simu-sensor && npm run test:coverage

# Watch mode (relance à chaque sauvegarde)
cd simu-sensor && npm run test -- --watch

# Un fichier spécifique
cd simu-sensor && npm test -- src/services/__tests__/
mqttService.test.ts
```

06 — Couverture Front Exploitation

Vue d'ensemble

Métrique	Valeur
Framework	Vue 3 + TypeScript + Vite
Runner	Vitest
Couverture mesurée	~97 % (<code>src/views</code> : 97,1 % par Vitest)
Fichiers de test	17 fichiers
Nombre de tests	165 tests
Chemin	<code>front-exploitation/src/tests/</code>
En CI	Oui job <code>front_exploitation_test</code>

Résultats par fichier

Fichier de test	Tests	Ce qui est couvert
<code>App.spec.ts</code>	12	Initialisation, routing, guards d'authentification
<code>components/DateRangePicker.spec.ts</code>	6	Sélecteur de plage de dates
<code>components/HumidityChart.spec.ts</code>	6	Graphique humidité (rendu, données vides)
<code>components/IncidentForm.spec.ts</code>	9	Formulaire déclaration incident (validation)
<code>components/LocaleSwitcher.spec.ts</code>	5	Changement de langue FR/EN
<code>components/QrCode.spec.ts</code>	6	Génération et affichage QR code

Fichier de test	Tests	Ce qui est couvert
components/ TemperatureChart.spec.ts	6	Graphique température (rendu, données vides)
views/ AlertesView.spec.ts	10	Liste alertes, filtres, résolution
views/ CapteursView.spec.ts	11	Vue capteurs — liste, statut, affectation entrepôt
views/ DashboardView.spec.ts	3	Dashboard — chargement des données
views/ EntrepotsListView.spec.ts	12	Liste entrepôts — tri, filtre actif
views/ EntrepotsView.spec.ts	10	Détail entrepôt — mesures, lots présents
views/ IncidentsView.spec.ts	15	CRUD incidents, types, filtres
views/ LotsView.spec.ts	15	Liste lots, déplacement, archivage
views/ ParametresView.spec.ts	9	Paramètres exploitation (seuils, nom)
views/ RolesView.spec.ts	16	Gestion des rôles et permissions
views/ UtilisateursView.spec.ts	14	CRUD utilisateurs, assignation de rôle
Total	165	

Points forts

- Toutes les vues principales ont des tests dédiés
- Les guards de rôle sont testés : un opérateur ne peut pas accéder aux paramètres
- Les états vides ("aucun lot", "aucune alerte") sont couverts

- Les graphiques sont testés avec des données réelles et des données vides
- 97 % de couverture sur `src/views` — le module le plus critique

Comment lancer

```
cd front-exploitation && npm install && npm test
cd front-exploitation && npm run test:coverage

# Un fichier spécifique
cd front-exploitation && npm test -- src/tests/views/
DashboardView.spec.ts
```

07 — Couverture Front Siège

Vue d'ensemble

Métrique	Valeur
Framework	Vue 3 + TypeScript + Vite
Runner	Vitest
Couverture mesurée	~95 % (33/33 fichiers couverts)
Fichiers de test	41 fichiers
Nombre de tests	250 tests
Chemin	front-siege/src/**/*.__tests__/ et front-siege/src/components/organisms/
En CI	Oui job front_siege_test

Résultats par dossier

Dossier	Fichiers de test	Tests
components/atoms/	4	23
components/molecules/	2	10
components/organisms/	5	33
components/__tests__/	6	33
services/__tests__/	6	27
stores/__tests__/	2	18
utils/__tests__/	1	11
pages/__tests__/	7	43
views/__tests__/	8	52
Total	41	250

Composants couverts

Composant	Chemin	Tests
Header.vue	components/organisms/	5
LoginForm.vue	components/organisms/	7
SiteForm.vue	components/organisms/	6
UserForm.vue	components/organisms/	9
Toast.vue	components/__tests__/	6
AdminSitesPage.vue	pages/__tests__/	6
AdminUsersPage.vue	pages/__tests__/	7
DashboardPage.vue	pages/__tests__/	7
LoginPage.vue	pages/__tests__/	4
ProfilePage.vue	pages/__tests__/	6
SiteDetailPage.vue	pages/__tests__/	6
SetPasswordPage.vue	pages/__tests__/	7

Stratégies de mock utilisées

- **Stores Pinia** : objets mutables au niveau module (`const mockAuthStore = { ... }`) modifiés dans `beforeEach` — évite les conflits de hoisting de `vi.mock()`
- **Vue Router** : mock complet de `useRoute` , `useRouter` , `RouterLink`
- **Remixicon** : stub de chaque icône en `<span />` pour éviter les erreurs d'import SVG
- **Composants enfants** : stub des `Header` , `FormField` , `Button` pour isoler la logique du composant testé

E2E Playwright

Un scénario E2E existe dans `front-siege/tests/e2e/admin-user-creation.spec.ts` . Il nécessite l'installation de `@playwright/test` :

```
cd front-siege && npm install --save-dev @playwright/test
```

Comment lancer

```
cd front-siege && npm install && npm test  
cd front-siege && npm run test:coverage  
  
# Un fichier spécifique  
cd front-siege && npm test -- src/pages/__tests__/  
SiteDetailPage.test.ts
```

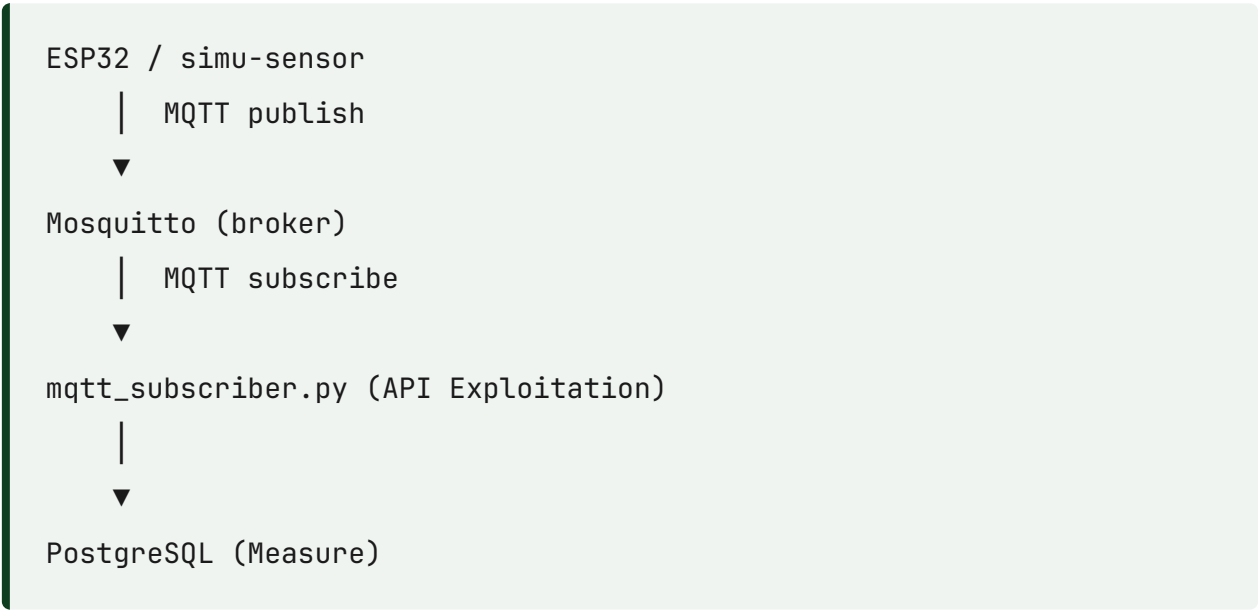
08 — Couverture Broker MQTT

Vue d'ensemble

Métrique	Valeur
Technologie	Mosquitto 2.x (conteneurisé)
Type de test	Intégration shell — vérification des règles ACL
Couverture	100 % (8/8 scénarios ACL)
Fichier de test	broker/tests/test_acl.sh
En CI	Oui job broker_acl_test

Rôle du broker

Le broker Mosquitto est le **point central de toutes les communications IoT**. Il reçoit les mesures publiées par les capteurs (ESP32 ou simu-sensor) et les redistribue au `mqtt_subscriber.py` dans l'API Exploitation.



Ce que testent les règles ACL

Mosquitto est configuré avec des règles **ACL (Access Control List)** qui définissent qui peut publier et qui peut s'abonner à quels topics. Ces règles

protègent le broker contre : - Des publications non autorisées (capteurs pirates) - Des lectures non autorisées (écoute passive des données)

Les 8 scénarios testés

#	Action	Topic	Client	Résultat attendu
1	Publish	capteur/ simu-sensor/ temperature	Capteur (simu)	Oui Autorisé
2	Publish	capteur/ simu-sensor/ humidity	Capteur (simu)	Oui Autorisé
3	Publish	capteur/iot/ temperature	Capteur (ESP32)	Oui Autorisé
4	Publish	capteur/iot/ humidity	Capteur (ESP32)	Oui Autorisé
5	Subscribe	capteur/+/ temperature	Subscriber API	Oui Autorisé
6	Subscribe	capteur/+/ humidity	Subscriber API	Oui Autorisé
7	Publish	capteur/+/ temperature	Client non autorisé	Non Refusé
8	Subscribe	capteur/+/ humidity	Client non autorisé	Non Refusé

Fonctionnement du script de test

```
# broker/tests/test_acl.sh

# 1. Build l'image broker
docker build -t futurekawa-broker-test broker/

# 2. Lance le container
docker run -d --name broker-test futurekawa-broker-test

# 3. Pour chaque scénario :
```

```
# - Tente pub/sub avec mosquitto_pub / mosquitto_sub
# - Vérifie le code de retour (0 = succès, 5 = refusé)
# - Affiche PASS ou FAIL

# 4. Arrête et supprime le container de test
```

Lancer les tests broker

```
# Via le pipeline CI (automatique)
# Ou manuellement :
bash broker/tests/test_acl.sh
```

Pré-requis : Docker installé, clients MQTT (`mosquitto-clients`) disponibles.

Topics MQTT du projet

Définis dans `API-exploitation/mqtt_topics.py` :

Topic	Sens	Usage
<code>capteur/+/temperature</code>	Capteur → API	Mesure température (toute source)
<code>capteur/+/humidity</code>	Capteur → API	Mesure humidité (toute source)
<code>capteur/simu-sensor/temperature</code>	simu-sensor → API	Simulation São Paulo
<code>capteur/simu-sensor/humidity</code>	simu-sensor → API	Simulation São Paulo
<code>capteur/iot/temperature</code>	ESP32 → API	Capteur physique Quito
<code>capteur/iot/humidity</code>	ESP32 → API	Capteur physique Quito

Le payload JSON inclut un champ `sensor_id` qui permet au subscriber de router la mesure vers le bon entrepôt :

```
{  
  "value": 28.4,  
  "sensor_id": "capteur-iot-1",  
  "timestamp": "2026-07-01T19:34:00Z"  
}
```

09 — Couverture — IoT (ESP32)

Vue d'ensemble

Le composant IoT est un **capteur physique ESP32** (DHT22) qui publie des mesures de température et d'humidité sur le broker MQTT. Le code embarqué est en C/Arduino et ne peut pas être testé directement par un runner CI classique. Les tests IoT sont donc écrits en **Python (unittest)** et couvrent :

- La validation logique des données capteur (portée de `sensors.h` en Python)
- La structure des payloads JSON publiés sur MQTT
- Le format des topics MQTT
- Le comportement des scripts shell d'upload et de détection du port série

```
iot/
├── code/
│   ├── code.ino          ← Point d'entrée Arduino (loop, setup)
│   ├── sensors.h         ← Lecture DHT22 + validation des mesures
│   ├── mqtt_handler.h    ← Publication MQTT (payloads JSON, topics)
│   ├── wifi_handler.h    ← Connexion WiFi
│   └── config.h          ← Paramètres WiFi/MQTT/capteur
└── tests/
    ├── test_iot_scripts.py      ← Tests des scripts shell
    ├── test_sensor_measurements.py ← Tests de base des mesures
    └── test_sensor_validation.py ← Tests de validation DHT22
                                   (logique sensors.h)
        └── test_mqtt_payload.py ← Tests des payloads et topics
MQTT
```

Exécution

```
# Depuis la racine du projet
python3 -m unittest discover -s iot/tests -p 'test_*.py' -v
```

Résultats

Suite	Tests	Statut
Scripts shell	4	Oui
Mesures de base	4	Oui
Validation capteur (sensors.h)	16	Oui
Payload & topics MQTT	12	Oui
Total	36	Oui

La couverture IoT est **comportementale**, pas en lignes : le code C embarqué ne peut pas être instrumenté en CI. Chaque test valide une règle métier précise issue du code source C.

test_iot_scripts.py — Scripts shell (4 tests)

Teste les scripts shell qui pilotent l'upload sur l'ESP32.

Test	Script	Ce qui est vérifié
<code>test_find_port_optional_mode_skips_without_hardware</code>	<code>find_port.sh</code>	En mode optionnel (<code>IOT_REQUIRED=0</code>), absence de port → <code>[SKIP]</code> en stderr, code retour 0
<code>test_find_port_required_mode_fails_without_hardware</code>	<code>find_port.sh</code>	En mode requis (<code>IOT_REQUIRED=1</code>), absence de port → <code>[ERROR]</code> en stderr, code retour 1
<code>test_run_iot_upload_script_skips_when_no_port_is_detected</code>	<code>run_iot_upload.sh</code>	Sans port détecté, l'upload s'annule proprement (code retour 0)

Test	Script	Ce qui est vérifié
<code>test_ensure_port_free_without_argument_is_safe</code>	<code>ensure_port_free.sh</code>	Sans argument, aucune action dangereuse, retourne « No port specified »

Non couvert : `esp32-helper.sh` — fonctions interactives (`select`) non testables sans TTY.

`test_sensor_measurements.py` — Mesures de base (4 tests)

Tests de base sur les types et intervalles des valeurs reçues du capteur.

Test	Ce qui est vérifié
Température reçue en float	La valeur est de type numérique
Humidité reçue en float	La valeur est de type numérique
Lecture complète contient température + humidité	Les deux champs coexistent
Valeurs invalides identifiées comme hors limites	<code>temperature < 0</code> ou <code>humidity > 100</code> sont détectables

`test_sensor_validation.py` — Validation DHT22 (16 tests)

Reproduit en Python la logique de validation de `sensors.h`. Couvre les bornes exactes issues du code C :

```
température : temp > -20 && temp < 60   (bornes exclusives)
humidité    : humi ≥ 0 && humi ≤ 100   (bornes inclusives)
NaN         → invalide dans les deux cas
```

Température (10 tests)

Test	Valeur	Attendu
Nominale	24.7	Oui valide
Zéro	0.0	Oui valide
Négative dans la plage	-10.0	Oui valide

Test	Valeur	Attendu
En dessous de -20	-25.0	Non rejetée
À exactement -20	-20.0	Non rejetée (borne exclusive)
Juste au-dessus de -20	-19.9	Oui valide
Au-dessus de 60	65.0	Non rejetée
À exactement 60	60.0	Non rejetée (borne exclusive)
Juste en dessous de 60	59.9	Oui valide
NaN	<code>float('nan')</code>	Non rejetée

Humidité (6 tests)

Test	Valeur	Attendu
Nominale	65.3	Oui valide
À exactement 0	0.0	Oui valide (borne inclusive)
À exactement 100	100.0	Oui valide (borne inclusive)
En dessous de 0	-1.0	Non rejetée
Au-dessus de 100	110.0	Non rejetée
NaN	<code>float('nan')</code>	Non rejetée

Lecture combinée (6 tests)

Vérifie qu'une lecture complète n'est valide que si température ET humidité sont toutes les deux valides.

`test_mqtt_payload.py` — Payloads & topics MQTT (12 tests)

Valide la structure JSON publiée sur le broker et les noms de topics, en miroir de `mqtt_handler.h`.

Topics MQTT (4 tests)

Topic	Valeur attendue
Température	capteur/iot/temperature
Humidité	capteur/iot/humidity
Erreur	capteur/iot/error
Préfixe commun	Tous commencent par capteur/iot/

Payload température (5 tests)

Champs requis : `metric` , `value` , `timestamp` , `sensor_id` . Le champ `metric` vaut exactement `"temperature"` .

Payload humidité (3 tests)

Champs requis : `metric` , `value` , `timestamp` , `sensor_id` . Le champ `metric` vaut exactement `"humidity"` .

Payload erreur (3 tests)

Champs requis : `error_type` , `consecutive_errors` , `timestamp` , `sensor_id` , `message` . Le champ `message` vaut exactement `"Capteur DHT22 ne répond pas"` .